॥ श्रीगणेशाय नमः ॥

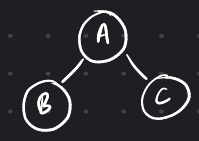
Binary Trees

A binary tree is a tree with atmost 2 children.

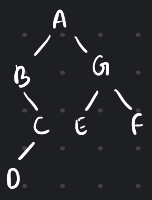
Search: generally means finding 1 element
Traversal: going through all elements

Tree traversals

In-order: left-root-right
Pre-order: root-left-right
Post-order: left-right-root



In-order : BAC
Pre-order : ABC
Post-order : BCA



In-order : B D C A E G F
Pre-order : A B C D G E F
Post-order : D C B E F G A



draw missing branches; makes it easier

In-order : B D A E C F
Pre-order : A B D C E F
Post-order : D B E F C A

Representing a binary tree node

```python
class BinaryTree:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

```c
struct node {
    int data;
    struct node *left;
    struct node *right;
};
```

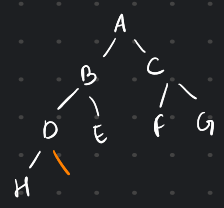
In-order, pre-order, and post-order traversals

```python
def in_order_traversal(tree):
    if tree is None:
        return
    in_order_traversal(tree.left)
    print(tree.value)
    in_order_traversal(tree.right)

def pre_order_traversal(tree):
    if tree is None:
        return
    print(tree.value)
    pre_order_traversal(tree.left)
    pre_order_traversal(tree.right)

def post_order_traversal(tree):
    if tree is None:
        return
    post_order_traversal(tree.left)
    post_order_traversal(tree.right)
    print(tree.value)
```

```c
void in_order(struct node *root) {
    if (root == NULL) {
        return;
    }
    in_order(root->left);
    printf("%d ", root->data);
    in_order(root->right);
}
```



In-order : HDBEAFCG
Pre-order : ABDHECFG
Post-order : HDEBFGCA

Q. double order traversal

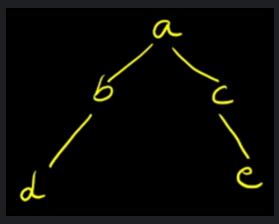
```
void double_order(struct node *t) {
    if (t) {
        printf("%c ", t->data);
        double_order(t->left);
        printf("%c ", t->data);
        double_order(t->right);
    }
}
```

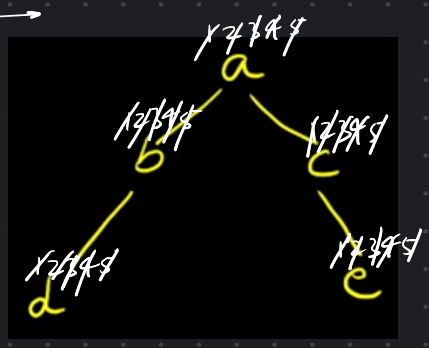


abbdffdaceeggc

Q. triple order traversal

```
void triple_order(struct node *t) {
    if (t) {
        printf("%c ", t->data);
        triple_order(t->left);
        printf("%c ", t->data);
        triple_order(t->right);
        printf("%c ", t->data);
    }
}
```



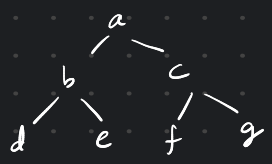


abdddbbaccceeeeca

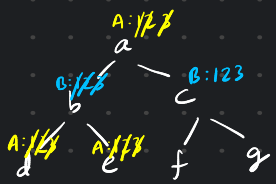
Q.

```
void A(struct node *t) {
    if (t) {
        printf("%c ", t->data);
        B(t->left);
        B(t->right);
    }
}
```

```
void B(struct node *t) {
    if (t) {
        A(t->left);
        printf("%c ", t->data);
        A(t->right);
    }
}
```



Output of A(root)?



adbe fcg
= by symmetry

For in-order, pre-order, and post-order traversals:
Time complexity: O(n) n nodes are visited
Space complexity: O(n) in the worst case, a skewed tree might be present



↳ avg case : $\Theta(\log n)$

of unlabeled binary trees possible using 3 nodes?



$\frac{^{2n}C_n}{n+1}$

of labeled binary trees possible using 3 nodes?



For each case, we get 3! ways.

$\frac{^{2n}C_n}{n+1} \cdot n!$

To construct a binary tree given a pre-order traversal



both have the same pre-order traversal a-b-c.

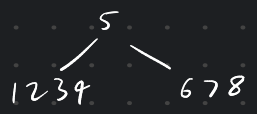
We can construct binary trees uniquely given their
in-order and pre-order
OR in-order and post-order traversals. but not using pre- and post- order traversals.

Q. In-order traversal: 1 2 3 4 5 6 7 8, pre-order traversal: 5 3 1 2 4 6 8 7.

→ In-order: left-root-right Pre-order: root-left-right

Pre: 5 3 1 2 4 6 8 7
↑ root

In: 1 2 3 4 5 6 7 8
root



Pre: 3 1 2 4 6 8 7
root root

In: 1 2 3 4 5 6 7 8



Pre: 1 2 4 8 7



Counting nodes in a BT

To count the # of nodes in a BT

root

#_of_nodes(tree) = 1 + #_of_nodes(left sub tree) + #_of_nodes(right sub tree) if the tree exists
0 if tree doesn't exist

```
def no_of_nodes(tree):
    return 1 + no_of_nodes(tree.left) + no_of_nodes(tree.right) if tree else 0
```

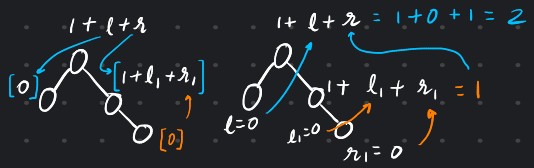
To count the # of leaves in a BT

```
def no_of_leaves(t):
    if t == None:
        return 0
    if t.left == None and t.right == None:
        return 1
    return no_of_leaves(t.left) + no_of_leaves(t.right)
```

To count the # of non-leaves in a BT

non_leaves(t) = 0 if t is a leaf
1 + non_leaves(left_st) + non_leaves(right_st) otherwise

```
def no_of_non_leaves(t):
    if t is None:
        return 0
    if t.left == None and t.right == None:
        return 0
    return 1 + no_of_non_leaves(t.left) + no_of_non_leaves(t.right)
```



To count the # of full nodes in a BT

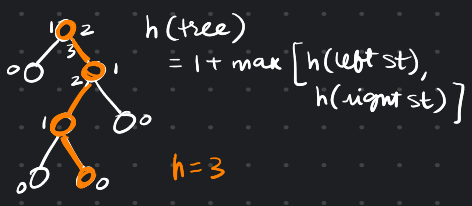
Full node: a node with the maximum # of children (2 for binary trees)

full_nodes(t) = 0 if t doesn't exist or is a leaf
1 + full_nodes(left_st) + full_nodes(right_st) if t is a full node
full_nodes(left_st) + full_nodes(right_st) if t is not full, but has only 1 child

```
def no_of_full_nodes(t):
    if t is None:
        return 0
    if t.left != None and t.right != None:
        return 1 + no_of_full_nodes(t.left) + no_of_full_nodes(t.right)
    return no_of_full_nodes(t.left) + no_of_full_nodes(t.right)
```

```
def full_nodes(t):
    if t is None:
        return 0
    return full_nodes(t.left) + full_nodes(t.right) + (1 if t.left and t.right else 0)
```

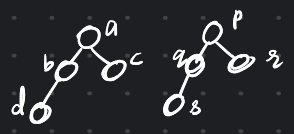
To find the height of a BT assuming the leaf is at height 0.




```
def height_of_tree(t):
    # assuming leaves are at height 0
    if t is None:
        return 0
    if t.left == None and t.right == None:
        return 0    # leaf
    return 1 + max(height_of_tree(t.left), height_of_tree(t.right))
```

To check if 2 BTs are identical

Traverse both sub-trees parallelly.



Both are identical iff $a == p$, $a \rightarrow left == p \rightarrow left$, $a \rightarrow right == p \rightarrow right$ (check conditions recursively).

```
def are_identical(t1, t2):
    if t1 == None and t2 == None:
        return True
    if t1 == None or t2 == None:
        return False
    return t1.value == t2.value\
            and are_identical(t1.left, t2.left)\
            and are_identical(t1.right, t2.right)
```

To get the mirror image of a BT

```
def mirror_image(t):
    if t is None:
        return t
    if t.left == None and t.right == None:
        return t

    temp = t.right
    t.right = mirror_image(t.left)
    t.left = mirror_image(temp)

    # or
    # t.left, t.right = mirror_image(t.right), mirror_image(t.left)

    return t
```

To check if 2 BTs are mirror images of one another

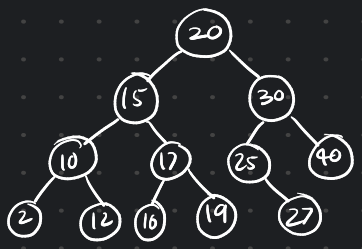
1. Take one of them, say t1, and find its mirror image, say m1.
2. Compare m1 and m2.

OR

```
def are_mirror(t1, t2):
    if t1 == None and t2 == None:
        return True
    if t1 == None or t2 == None:
        return False
    return t1.value == t2.value\
            and are_mirror(t1.left, t2.right)\
            and are_mirror(t1.right, t2.left)
```

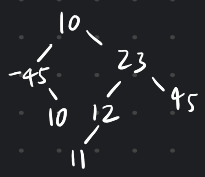
Binary search trees

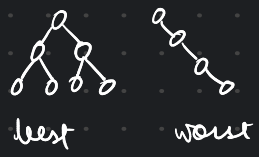
A BST is a BT in which at every node, the left sub-tree has values less-than and the right sub-tree has values greater than that in the node.



For a balanced tree, the time complexity to search a node is T(n) = O(logn) (average/best case).
For a skewed tree, it may go to O(n) (worst case).

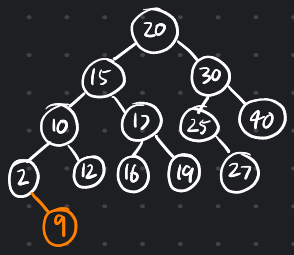
10 23 45 -45 12 11 10





To construct a BST, time complexity is
- Best case: O(nlogn)
- Worst case: $O(n^2)$.

To insert into a BST:



Insert 9.

Time complexity:
- Average case: O(logn)
- Worst case: O(n)
- Best case: O(1)

To delete from a BST:

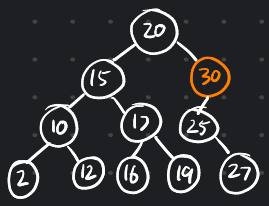


To delete 40:

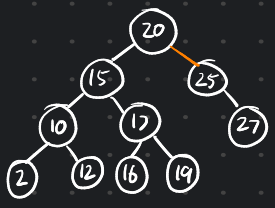
set right pointer of parent to NULL and free memory.



If node is leaf

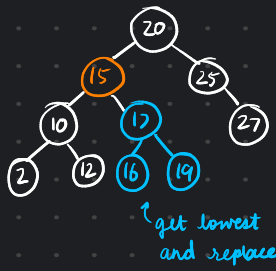


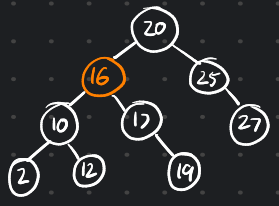
connect child to parent



If node has 1 child

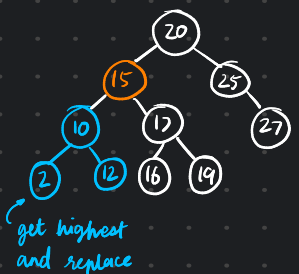






If node has 2 children

OR





In-order traversal of a BST gives the elements in a sorted order.



02 10 12 15 16 17 19 20 25 27 30 40

When deleting an element, we replace it with its in-order successor or predecessor.

To find those, keep going right in LST or left in RST.

Deletion of a node:
Find node x to delete O(n) worst
If it has no children, simply remove it.
If it has one children, connect its child to its parent.
If it has 2 children, delete the node and replace it with its in-order predecessor or successor. O(n) worst

T = O(n) worst

To find the min element in a BST
Keep going to the left until there's any left. T = O(n) (height)

```
def find_min(bst):
    while bst.left:
        bst = bst.left

    return bst.value
```

Q. How many BSTs are possible with 4 distinct keys?

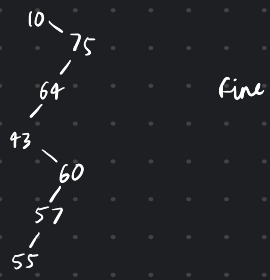
$\frac{2n C_n}{n+1}$



← For n = 3

Q. [1, ..., 100] are put into a BST. Which of these cannot be a sequence of nodes visited when searching for 55?

a. 10, 75, 64, 43, 60, 57, 55



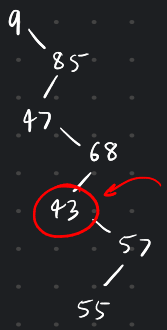
Fine

b. 90, 12, 68, 34, 62, 45, 55



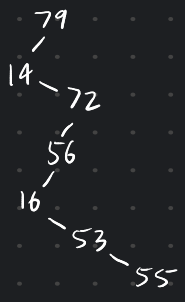
Fine

c. 9, 85, 47, 68, 43, 57, 55



cannot be present to the right of 47

d. 79, 14, 72, 56, 16, 53, 55



Fine

(check if we always go in the expected direction of 55 and that we always get a BST).

8.4.30 Binary Search Tree: GATE IT 2006 | Question: 45

Suppose that we have numbers between 1 and 100 in a binary search tree and want to search for the number 55. Which of the following sequences CANNOT be the sequence of nodes examined?

A. $\{10, 75, 64, 43, 60, 57, 55\}$
B. $\{90, 12, 68, 34, 62, 45, 55\}$
C. $\{9, 85, 47, 68, 43, 57, 55\}$
D. $\{79, 14, 72, 56, 16, 53, 55\}$

gateit-2006 data-structures binary-search-tree normal

8.5.51 Binary Tree: GATE IT 2008 | Question: 46

The following three are known to be the preorder, inorder and postorder sequences of a binary tree. But it is not known which is which.

I. $MBCAFHPYK$
II. $KAMCBYPFH$
III. $MABCKYFPH$

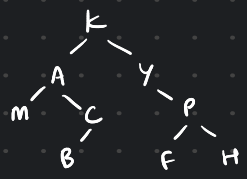
Pick the true statement from the following.

A. I and II are preorder and inorder sequences, respectively
B. I and III are preorder and postorder sequences, respectively
C. II is the inorder sequence, but nothing more can be said about the other two sequences
D. II and III are the preorder and inorder sequences, respectively

Pre-order : root at beginning, post-order : root at end.

I. $MBCAFHPYK$ — post
II. $KAMCBYPFH$ — pre
III. $MABCKYFPH$ — in-order

To verify: use in- and pre-order to construct tree and check post-order.



8.4.32 Binary Search Tree: GATE IT 2008 | Question: 71

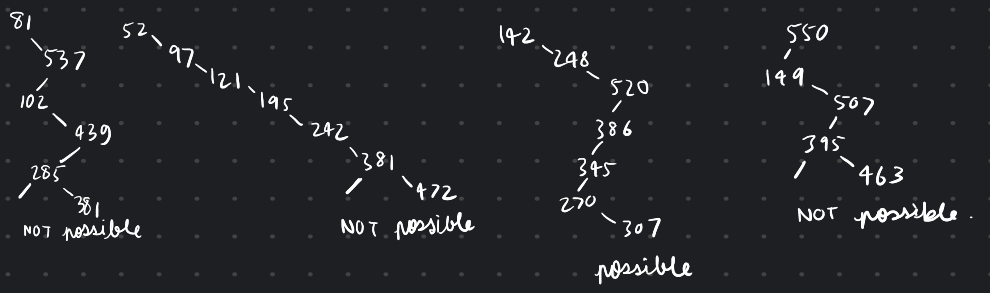
A Binary Search Tree (BST) stores values in the range 37 to 573. Consider the following sequence of keys.

I. $81, 537, 102, 439, 285, 376, 305$
II. $52, 97, 121, 195, 242, 381, 472$
III. $142, 248, 520, 386, 345, 270, 307$
IV. $550, 149, 507, 395, 463, 402, 270$

Suppose the BST has been unsuccessfully searched for key 273. Which all of the above sequences list nodes in the order in which we could have encountered them in the search?

A. II and III only B. I and III only C. III and IV only D. III only

gateit-2008 data-structures binary-search-tree normal



8.13.5 Tree: GATE CSE 1998 | Question: 2.11

A complete n-ary tree is one in which every node has 0 or n sons. If x is the number of internal nodes of a complete n-ary tree, the number of leaves in it is given by

A. $x(n-1)+1$ B. $xn-1$ C. $xn+1$ D. $x(n+1)$

gate1998 data-structures tree normal

e.g. 3-ary
$n=3$



$x=3$

leaves $=7$

$7 = 3\cdot(3-1)+1$

8.13.7 Tree: GATE CSE 2002 | Question: 2.9

The number of leaf nodes in a rooted tree of n nodes, with each node having 0 or 3 children is:

A. $\frac{n}{2}$ B. $\frac{(n-1)}{3}$ C. $\frac{(n-1)}{2}$ D. $\frac{(2n+1)}{3}$

gatecse-2002 data-structures tree normal



nodes $=7$
leaves $=5$

$\frac{2(7)+1}{3} = 5$

8.5.15 Binary Tree: GATE CSE 1995 | Question: 1.17

A binary tree T has n leaf nodes. The number of nodes of degree 2 in T is

A. $\log_2 n$ B. $n-1$ C. n D. 2^n

gate1995 data-structures binary-tree normal



$n=6$ 5 internal nodes

8.5.50 Binary Tree: GATE IT 2006 | Question: 9

In a binary tree, the number of internal nodes of degree 1 is 5, and the number of internal nodes of degree 2 is 10. The number of leaf nodes in the binary tree is

A. 10 B. 11 C. 12 D. 15

gateit-2006 data-structures binary-tree normal



of leaves = # of degree 2 nodes + 1

Balanced BSTs
- AVL trees
- Red-black trees
- 2-3 trees

AVL trees
- search time: O(logn)
- insertion: O(logn)
- deletion: O(logn)
- balancing: O(logn)

8.1.4 AVL Tree: GATE IT 2008 | Question: 12

Which of the following is TRUE?

A. The cost of searching an AVL tree is $\Theta(\log n)$ but that of a binary search tree is $O(n)$
B. The cost of searching an AVL tree is $\Theta(\log n)$ but that of a complete binary tree is $\Theta(n \log n)$
C. The cost of searching a binary search tree is $O(\log n)$ but that of an AVL tree is $\Theta(n)$
D. The cost of searching an AVL tree is $\Theta(n \log n)$ but that of a binary search tree is $O(n)$

gateit-2008 data-structures binary-search-tree easy avl-tree

8.13.4 Tree: GATE CSE 1998 | Question: 1.24

Which of the following statements is false?

A. A tree with a n nodes has $(n-1)$ edges
B. A labeled rooted binary tree can be uniquely constructed given its postorder and preorder traversal results.
C. A complete binary tree with n internal nodes has $(n+1)$ leaves.
D. The maximum number of nodes in a binary tree of height h is $2^{h+1}-1$

gate1998 data-structures tree multiple-selects normal

GATE IT 2005 | Question: 50

asked Nov 3, 2014 • edited Dec 25, 2017 by **Kenzou**

26,621 views

64

In a binary tree, for every node the difference between the number of nodes in the left and right subtrees is at most 2. If the height of the tree is $h > 0$, then the minimum number of nodes in the tree is

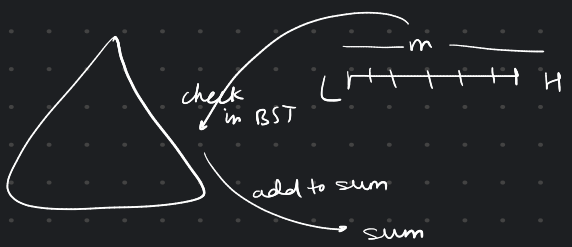
A. 2^{h-1}
B. $2^{h-1}+1$
C. $2^h - 1$
D. 2^h

DS #gateit-2005 #data-structures #binary-tree #normal

8.4.16 Binary Search Tree: GATE CSE 2014 Set 3 | Question: 39

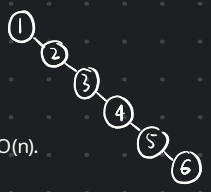
Suppose we have a balanced binary search tree T holding n numbers. We are given two numbers L and H and wish to sum up all the numbers in T that lie between L and H. Suppose there are m such numbers in T. If the tightest upper bound on the time to compute the sum is $O(n^a \log^b n + m^c \log^d n)$, the value of $a + 10b + 100c + 1000d$ is _____.

gatecse-2014-set3 data-structures binary-search-tree numerical-answers normal



In-order takes O(n)
gives sorted list
Apply binary search O(logn) to find L & M
To add up the m elements
O(n + logn + m)

AVL trees

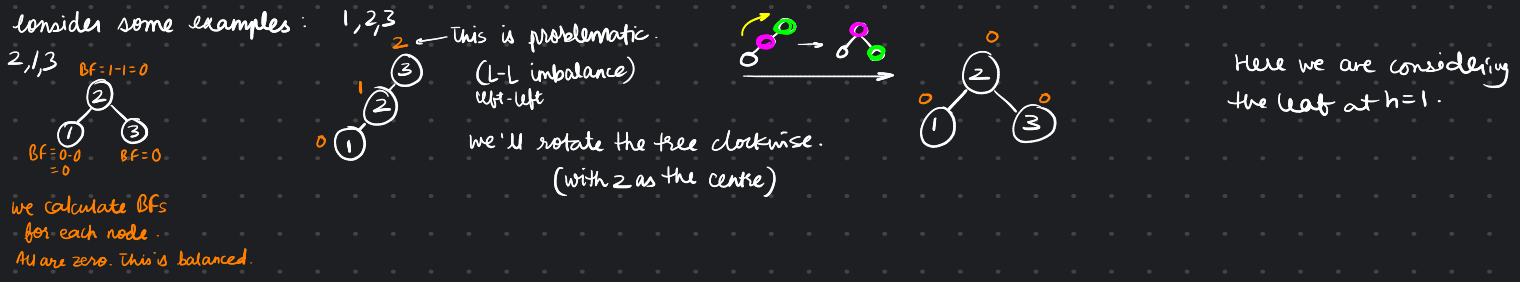


A BST could get really skewed. Say we insert elements in the order 1, 2, 3, 4, 5, 6, ...
The time complexity for searching would just go to O(n).
Instead, we could re-balance the tree. AVL trees are a type of balanced BSTs.
For an AVL tree, the height is always O(logn), as supposed to ordinary BSTs, for which the height could go upto O(n).

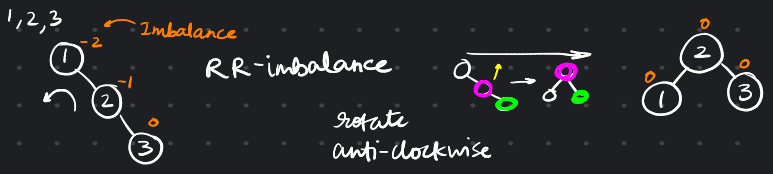
Balance factor of a node

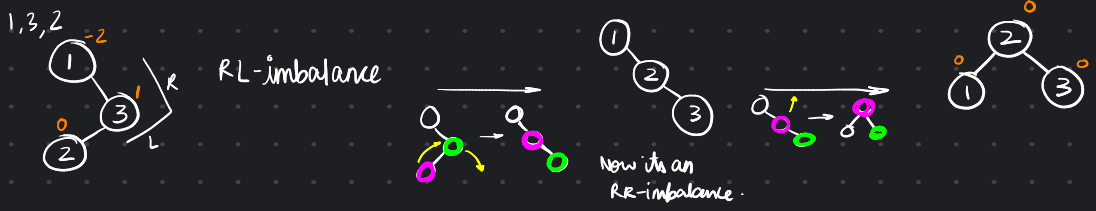
BF = (height of left sub-tree) - (height of right sub-tree)

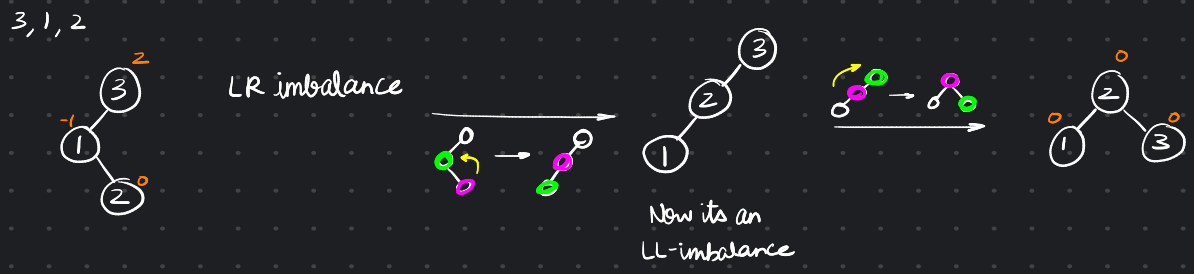
AVL trees permit a BF in {-1, 0, 1}. For any other BF (unbalanced tree), we'll rebalance the tree.



Every time we insert a node into an AVL tree, we check every node from that new node to root for imbalance and re-balance the tree accordingly.

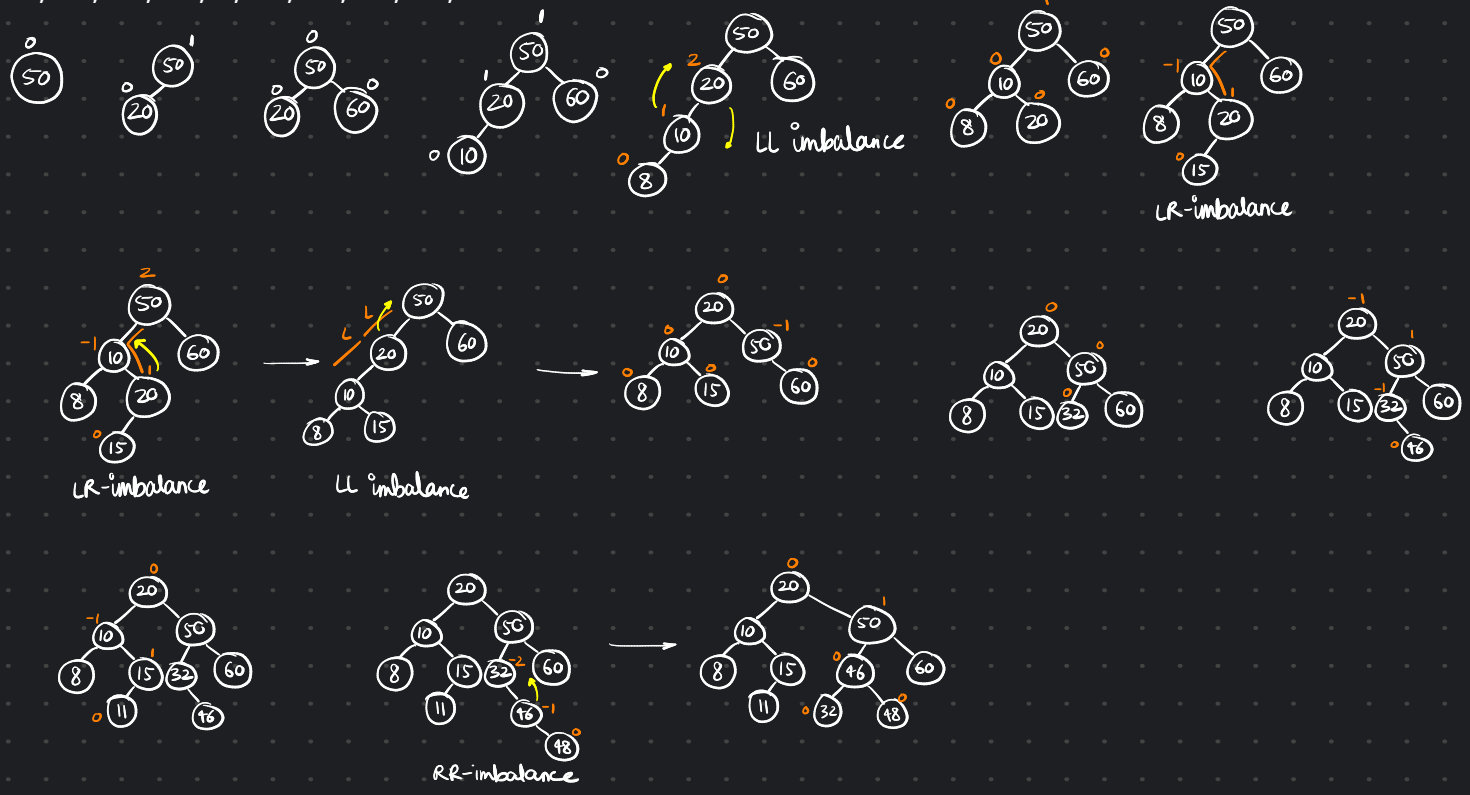






Building an AVL tree

50, 20, 60, 10, 8, 15, 32, 46, 11, 48



The resulting tree is a balanced tree (AVL).



Time complexity

	BST	AVL
search	O(n) at worst O(logn) avg. case	O(logn) always
height	O(n) at worst O(logn) avg. case	O(logn) always
insertion	O(n) at worst O(logn) avg.	O(logn) ← insertion + O(logn) ← to find imbalance + O(1) ← rotation = O(logn)

Best answer

175

In worst case for finding L and H it will take $O(\log n)$ time as the given tree is **balanced** binary search tree.
Now there are m elements between L and H. So to traverse m element it will take $O(m)$ time (traversal algorithm given below). So, total

$$O(m + \log n) \implies a = 0, b = 1, c = 1, d = 0$$
$$\therefore 0 + (10 \times 1) + (100 \times 1) + (1000 \times 0) = 110.$$

To find all the numbers from L to H we can do an inorder traversal from root and discard all elements before L and after H. But this has $O(n)$ time complexity. So, we can do a modification to inorder traversal and combine with binary search as follows:

1. Find L using binary search and keep all nodes encountered in the search in a stack.
2. After finding L add it to stack as well and initialize $\text{sum} = 0$.
3. Now, for all nodes in stack, do an inorder traversal starting from their right node and adding the node value to sum. If H is found, stop the algorithm.

answered Jan 3, 2015 • edited Jun 18, 2021 by **Lakshman Bhaiya**

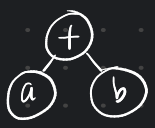
Comment Share Follow



Kalpish

Expression tree

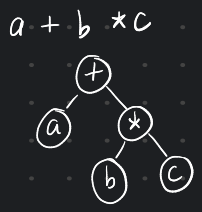
Internal nodes are operators, leaves are operands.

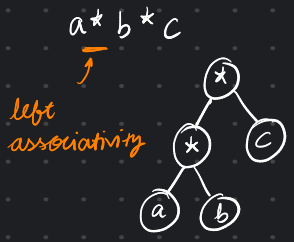


pre-order : +ab prefix expression

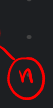
In-order : a+b

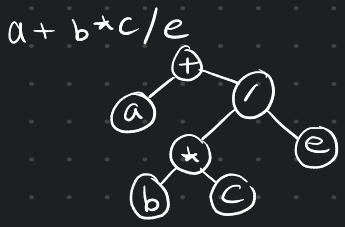
post-order : ab+







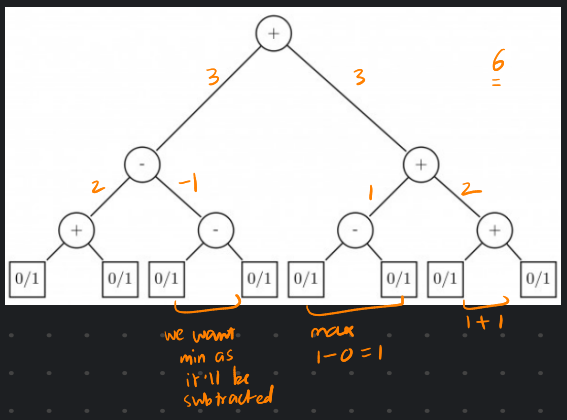
NOTE:  is NOT right as the in-order traversal gives !n



3.4.2 Expression Evaluation: GATE CSE 2014 Set 2 | Question: 39

Consider the expression tree shown. Each leaf represents a numerical value, which can either be 0 or 1. Over all possible choices of the values at the leaves, the maximum possible value of the expression represented by the tree is ___.

gatecse-2014-set2 compiler-design normal expression-evaluation numerical-answers

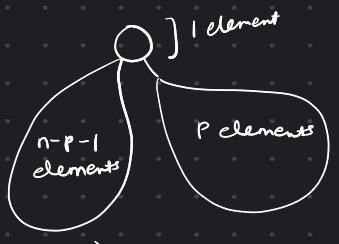


8.4.28 Binary Search Tree: GATE IT 2005 | Question: 12

The numbers $1, 2, \ldots . n$ are inserted in a binary search tree in some order. In the resulting tree, the right subtree of the root contains p nodes. The first number to be inserted in the tree must be

A. p B. $p+1$ C. $n-p$ D. $n-p+1$

gateit-2005 data-structures normal binary-search-tree



root = (n−p−1)+1 > n−p−1 elements

= n−p